Servomotores da Série M17 - FICHA TÉCNICA

Controlo de Movimento Tudo-em-Um, Simples e Acessível

Da Educação à Inovação



Introdução

Os Servomotores da Série M17 são soluções de controlo de movimento tudo-em-um que integram um motor, um driver de motor, um controlador de movimento e um encoder num único conjunto compacto. Estes servomotores dispõem de uma interface de comunicação RS-485, permitindo ligar várias unidades em cadeia (daisy-chain) e controlá-las a partir de um único ponto de ligação. Disponível em quatro modelos - M17-60, M17-48, M17-40 e M17-34 - a série oferece opções flexíveis para corresponder a requisitos específicos de binário (torque) e de tamanho, mantendo características de controlo consistentes. Cada Servomotor da Série M17 inclui funcionalidades de controlo sofisticadas, incluindo múltiplos modos de operação, capacidades de autocalibração e monitorização de estado integrada através de LEDs indicadores. Os motores podem ser facilmente controlados a partir de qualquer plataforma, incluindo Mac, PC, Raspberry Pi ou Arduino (necessitando apenas de um adaptador RS485 de baixo custo), o que os torna ideais tanto para aplicações educativas como industriais. A série suporta controlo de posição preciso com perfis de movimento trapezoidais, modo de controlo em malha fechada e tratamento abrangente de erros. Com dimensões de montagem NEMA 17 normalizadas, uma ampla gama de tensão (12-24V) e um protocolo de comunicação robusto, os Servomotores da Série M17 proporcionam uma solução fiável e flexível para aplicações que exigem controlo de movimento preciso, desde a robótica e as máquinas CNC até equipamentos de teste automatizados e instrumentos científicos.

Envie-nos o Seu Feedback

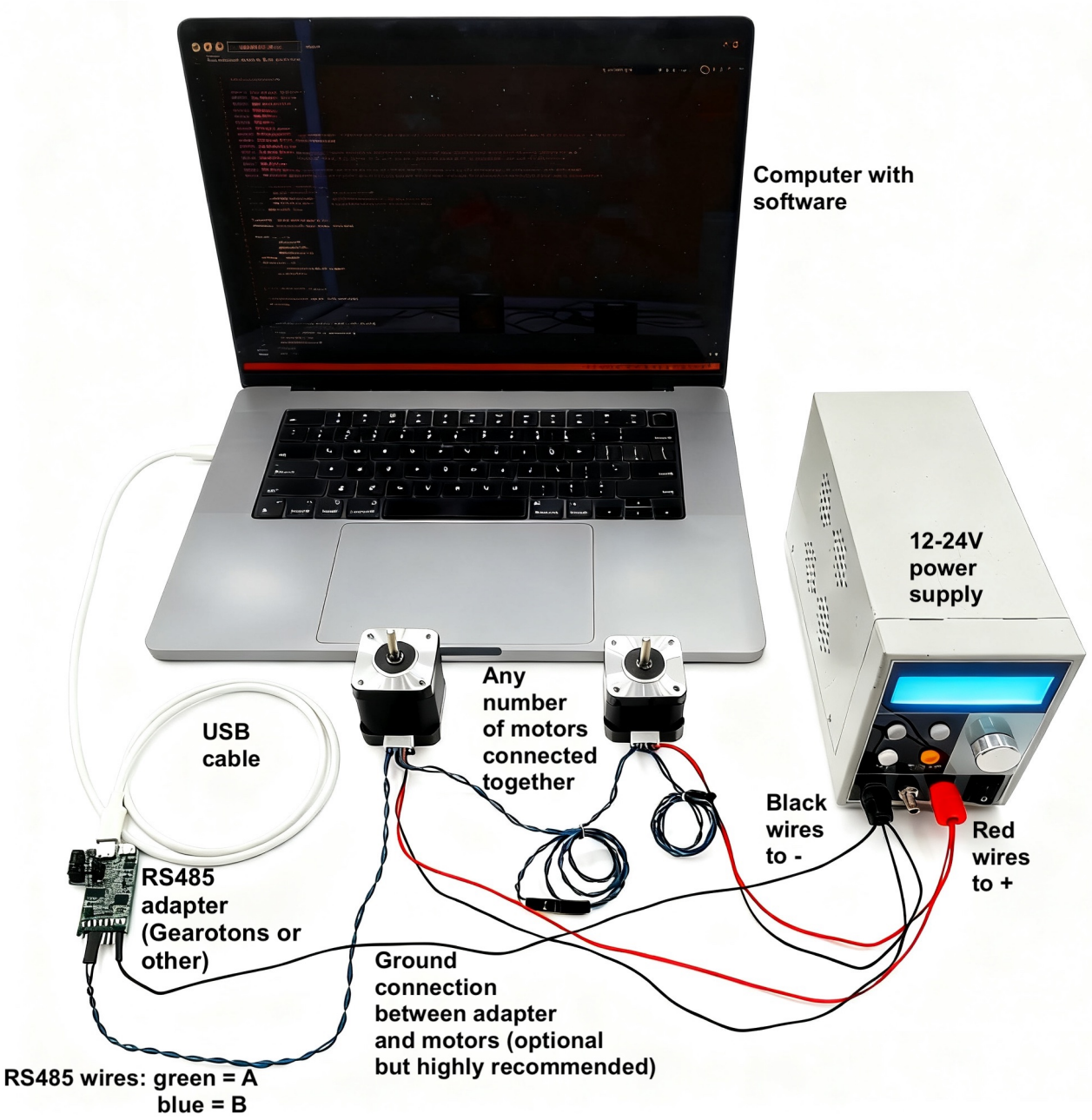
Se encontrar erros neste documento ou tiver questões, informe-nos em: tutorial.gearotons.com/feedback. Agradecemos muito as suas sugestões sobre como podemos melhorar o produto ou a documentação. Muito obrigado!

Visite a nossa Página de Feedback

Características Principais

- Elevado nível de integração, combinando um motor, um driver de motor, um sistema de controlo de movimento e um encoder
- Controle qualquer número de motores a partir de um único controlador simples através de uma interface de comunicação RS-485
- Controlo através de comandos de alto nível como "Enable mosfets" (ativar MOSFETs) e "Trapezoid move" (movimento trapezoidal), evitando o controlo DIR/STEP com temporização crítica
- Formato compacto, praticamente do mesmo tamanho de um motor de passo NEMA 17 com as mesmas especificações (sem protuberâncias)
- Dimensões de montagem NEMA 17 normalizadas
- Ampla gama de tensão (12-24V) para opções de alimentação flexíveis
- Controlo em malha fechada de alta precisão, com encoder integrado e malha de controlo PID a funcionar a 32 kHz
- Muito mais eficiente em termos energéticos em controlo de malha fechada do que um motor de passo convencional
- Proteção integrada contra sobrecorrente, sobretensão e sobreaquecimento
- A velocidade máxima pode atingir 560 RPM
- Relação binário/peso igual à de um motor de passo equivalente
- Compatível com uma grande variedade de interfaces e hardware, como Raspberry Pi, Arduino, ESP32, Mac e PC
- Esforçamo-nos por fornecer excelente documentação e tutoriais para o pôr a funcionar rapidamente
- Fornecemos documentação adequada para IA, caso queira que a sua IA favorita faça todo o trabalho por si
- Adequado para robótica, CNC, automação, instrumentos científicos, dispositivos de teste, impressoras 3D e muito mais
- Disponível em diferentes tamanhos, para que possa encontrar o binário e o preço certos para a sua aplicação

Diagrama de Ligações



Sistema de Unidades

Os Servomotores da Série M17 utilizam determinadas unidades internas para poderem efetuar de forma eficiente os cálculos associados ao movimento (usando aritmética de números inteiros). É da responsabilidade do software de controlo suportar múltiplas unidades de medida para as várias grandezas. As nossas bibliotecas Python e Arduino tratam automaticamente das conversões de unidades, permitindo-lhe trabalhar com as unidades da sua preferência. Abaixo encontram-se as unidades suportadas para cada grandeza:

Grandeza	Unidades Disponíveis
Tempo	seconds, milliseconds, minutes, microseconds, timesteps
Posição	shaft rotations, degrees, radians, encoder counts
Velocidade	rotations per second, rpm, degrees per second, radians per second, counts per second, counts per timestep
Aceleração	rotations per second squared, rpm per second, degrees per second squared, radians per second squared, counts per second squared, counts per timestep squared
Corrente	internal current units, milliamps, amps
Tensão	volts, millivolts
Temperatura	celsius, fahrenheit, kelvin

Guia de Iniciação

Para o ajudar a começar a utilizar o seu Servomotor da Série M17, disponibilizamos um guia online completo que cobre tudo, desde a configuração inicial até implementações avançadas do protocolo. Este guia inclui:

- Instruções de configuração passo a passo
- Documentação detalhada do protocolo de comunicação
- Exemplos de programação e excertos de código
- Descrição dos códigos de erro
- Dicas de resolução de problemas e boas práticas

Clique aqui para o Guia de Iniciação: tutorial.gearotons.com

LEDs Indicadores e Botões

O servomotor tem dois LEDs de estado (verde e vermelho). O LED verde pisca lentamente para mostrar um sinal de vida (heartbeat), pisca rapidamente para indicar que esta a ser executado o bootloader em vez da aplicacao e acende-se tambem brevemente enquanto um pacote esta a ser recebido, pelo que cintila com a comunicacao no barramento. O LED vermelho indica codigos de erro fatais piscando um determinado numero de vezes.

O servomotor tem dois botões identificados como "Reset" (reiniciar) e "Test" (testar). O botão Reset reinicia o microcontrolador interno e todo o estado regressa aos valores predefinidos. O botão Test faz o motor rodar. Prima brevemente para o fazer rodar num sentido e prima durante mais de 0,3 segundos e solte para o fazer rodar no sentido oposto. Mantenha premido durante pelo menos 2 segundos e solte para colocar o motor em modo de malha fechada. Mantenha premido durante mais de 15 segundos e solte para que o motor execute uma calibração de si próprio. Tenha em atenção que o motor roda durante a calibração e tem de poder rodar livremente para que a calibração seja bem-sucedida, pelo que deve remover quaisquer cargas antes de efetuar esta operação.

Visão Geral do Protocolo de Comunicação

Os Servomotores da Série M17 utilizam RS-485 para a comunicação, permitindo ligar vários motores em cadeia (daisy-chain). Cada motor tem um endereço único de 64 bits atribuído de fábrica, para que possa ser endereçado individualmente num barramento RS-485. A cada motor pode também ser atribuído um alias único de 1 byte (0-251), de forma a poupar bytes no pacote de comunicação e ainda assim obter um controlo individual semelhante. Existe também um alias de difusão (broadcast), que é o 255. O protocolo suporta um conjunto abrangente de comandos para controlo de movimento, configuração e monitorização de estado. A atualização de firmware através da interface RS-485 é suportada. A velocidade de comunicação está fixada em 230400 baud.

Resumo de Referência de Comandos

Para a fonte de informação atualizada sobre todos os comandos disponíveis, pode consultar este documento.

 https://github.com/tomrodinger/servomotor/blob/main/python_programs/servomotor/motor_commands.json

Também pode executar este comando:

```
pip3 install servomotor   # run this just once to install the library and programs
servomotor_command -c
```

Isto imprime a informação contida no ficheiro motor_commands.json de uma forma mais legível e fornece algumas indicações de utilização para enviar comandos ao motor a partir da linha de comandos.

Controlo Básico

Comando	Descrição
Disable MOSFETs	Desativa imediatamente as saidas do driver do motor (MOSFETs); o comando executa de imediato (nao e colocado na fila) e e idempotente.
Enable MOSFETs	Ativa as saidas do driver do motor (MOSFETs).
Reset time	Zere o relógio absoluto de microssegundos do dispositivo; como efeito secundário importante, toda a fila de movimentos é esvaziada e o motor para imediatamente.
Emergency stop	Pare o motor imediatamente: desative os MOSFETs, esvazie a fila de movimentos, zere a velocidade atual e ajuste a referência interna de posição para a posição atual.
Zero position	Define a posição atual como a posição zero (a origem).
System reset	Reinicia o dispositivo.

Controlo de Movimento

Comando	Descrição
Trapezoid move	Percorre o deslocamento com sinal indicado ao longo exatamente da duracao indicada, relativamente a posicao no fim do movimento previamente colocado na fila (para um alvo absoluto, utilize 'Go to position').
Go to position	Coloca na fila um movimento absoluto suave.
Homing	Efetua o retorno a origem (homing) do motor, movendo-o ate atingir um batente rigido.
Go to closed loop	Entra no modo de controlo de posicao em malha fechada.
Move with acceleration	Coloca na fila um segmento de aceleracao constante: a aceleracao e aplicada uma vez por passo de tempo, durante o numero de passos de tempo indicado.
Move with velocity	Roda o motor a uma velocidade constante durante uma duracao fixa.
Multimove	Coloca na fila ate 32 movimentos num unico comando; cada movimento e um segmento de aceleracao ou uma alteracao instantanea de velocidade (selecionado por movimento atraves da mascara de bits moveTypes), executado durante o seu numero indicado de passos de tempo.

Configuração

Comando	Descrição
Set maximum velocity	Define a velocidade maxima permitida.
Set maximum acceleration	Define a aceleracao maxima utilizada para planear todos os movimentos trapezoidais colocados na fila posteriormente ('Go to position', 'Trapezoid move', movimentos de calibracao); os itens ja presentes na fila nao sao afetados.
Start calibration	Executa uma calibracao completa do motor.
Set maximum motor current	Define a corrente maxima do motor e a corrente maxima de regeneracao.
Set safety limits	Define os limites de seguranca de posicao.
Test mode	Define ou aciona um modo de teste.
Set PID constants	Define as constantes P, I e D do controlador em malha fechada que tenta manter a trajetoria de movimento.
Set max allowable position deviation	Defina a distância máxima que a posição real do motor (medida pelos sensores Hall) pode se desviar da posição comandada; excedê-la gera o erro fatal 45 (ERROR_POSITION_DEVIATION_TOO_LARGE).
CRC32 control	Ativa ou desativa a camada CRC32 do protocolo para este dispositivo, em ambas as direcoes.
Get PID constants	Leia um instantâneo consistente das constantes P, I e D brutas configuradas.

Estado e Monitorização

Comando	Descrição
Get current time	Retorne o tempo absoluto do dispositivo como uma contagem de 64 bits sem sinal dos microssegundos reais decorridos desde a energização ou o último 'Reset time'; não é a hora do calendário.
Get n queued items	Retorne o número de itens na fila de movimentos como um único byte entre 0 e 32 (a fila comporta no máximo 32 itens).
Get hall sensor position	Obtem a posicao medida pelos sensores de Hall, que e a posicao real do veio.
Get status	Obtem os sinalizadores de estado e o codigo de erro fatal do motor.
Control hall sensor statistics	Liga ou desliga a recolha de estatisticas dos sensores de Hall.
Get hall sensor statistics	Le as estatisticas recolhidas dos tres sensores de Hall analogicos.
Get position	Obtem a posicao comandada (desejada) atual, ou seja, a posicao-alvo interna do perfil de movimento que avanca a cada ciclo de controlo.
Get comprehensive position	Obtem a posicao comandada, a posicao (medida) dos sensores de Hall e a contagem do encoder externo, tudo numa unica leitura.
Get supply voltage	Obtém a tensão medida da fonte de alimentação.
Get max PID error	Obtem o erro de posicao minimo e maximo observado pelo controlador PID em malha fechada desde a ultima leitura e, em seguida, repoe os acumuladores.
Get temperature	Obtem a temperatura medida por um sensor analogico dedicado na PCB do driver do motor (nao a pastilha do microcontrolador nem os enrolamentos do motor).
Get debug values	Leia um instantâneo de vários valores de diagnóstico internos.
Get communication statistics	Obtem seis contadores de erros de comunicacao e, opcionalmente, repoe-nos.

Gestão de Dispositivos

Comando	Descrição
Time sync	Envie a leitura do relógio do mestre ao motor para que ele possa ajustar sua frequência de clock.
Get product specs	Obtem duas constantes do produto necessarias para os calculos de movimento.
Detect devices	Deteta todos os dispositivos ligados a interface RS485.
Set device alias	Define o alias de um byte do dispositivo, guarda-o na flash nao volatil e, em seguida, reinicia automaticamente o dispositivo.
Get product info	Obtem as informacoes do produto: codigo do produto (numero do modelo), codigo de compatibilidade do firmware, versao do hardware, numero de serie e ID unico, devolvidos como um payload de resposta de 28 bytes.
Firmware upgrade	Escreve uma pagina de 2048 bytes de firmware da aplicacao na flash.
Get product description	Obtem uma cadeia de texto curta com a descricao do produto.
Get firmware version	Obtem a versao do firmware ou a versao do bootloader, se o dispositivo estiver atualmente a executar o bootloader.
Ping	Envia um payload de 10 bytes e o dispositivo responde imediatamente com os mesmos 10 bytes (executa de imediato, nao e colocado na fila).
Vibrate	Faça o motor vibrar (implementado somente no M1; sem efeito no M2, M17 e M23) alternando rapidamente a tensão PWM em malha aberta entre +50 e -50 (amplitude fixa codificada), ou interrompa a vibração.
Identify	Identifica um motor fazendo piscar rapidamente o seu LED verde: 30 flashes de cerca de 90 ms cada, aproximadamente 2,7 segundos no total, apos o que o piscar normal do sinal de vida (heartbeat) de 1 segundo e retomado.

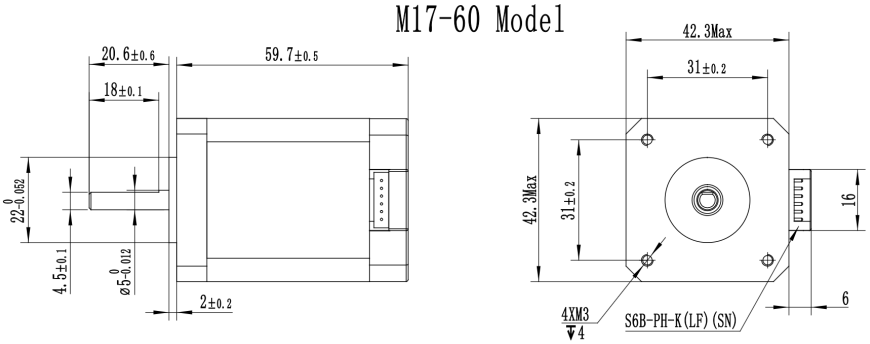
Outros

Comando	Descrição
Capture hall sensor data	Captura dados dos sensores de Hall e transmite-os de volta.
Read multipurpose buffer	Le e limpa o buffer de dados multiusos, que contem no maximo um conjunto de dados de cada vez.

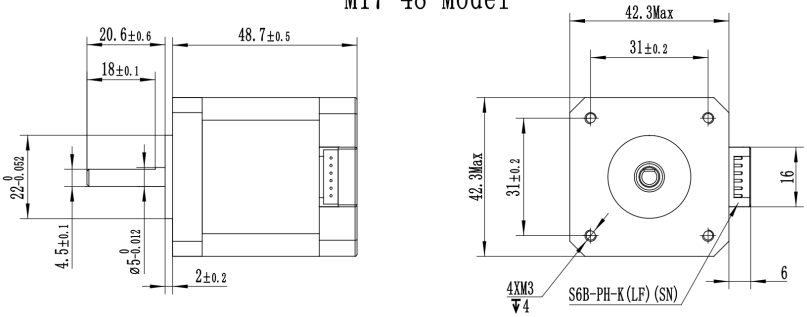
Especificações Mecânicas

Parâmetro	M17-60	M17-48	M17-40	M17-34
Dimensões (CxL)	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm
Altura	59.7 mm	48.7 mm	40.1 mm	33.5 mm
Comprimento do Veio	20.6 mm	20.6 mm	20.6 mm	20.6 mm
Peso	470g	360g	285g	210g
Classe de Proteção	IP20	IP20	IP20	IP20

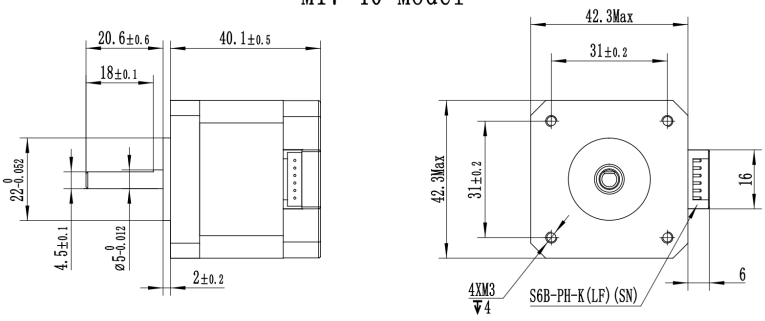
M17-60 Model



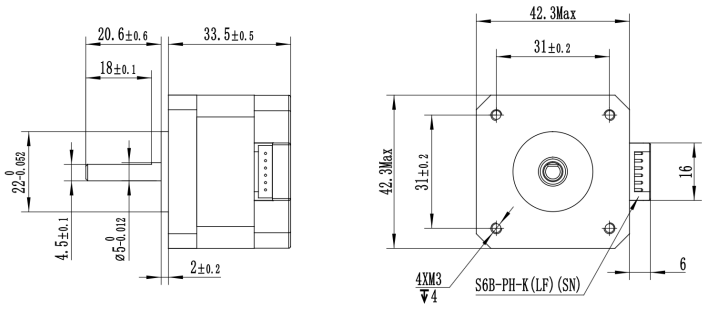
M17-48 Model



M17-40 Model



M17-34 Model



Especificações Técnicas

Parâmetro	M17-60	M17-48	M17-40	M17-34
Tensão de Funcionamento	12-24V	12-24V	12-24V	12-24V
Binário Nominal	0.65 N.m	0.55 N.m	0.42 N.m	0.28 N.m
Velocidade Máxima	560 RPM	560 RPM	560 RPM	560 RPM
Corrente Máxima	1.1A	1.1A	1.1A	1.0A
Potência Nominal	26.4W	26.4W	26.4W	24.0W

Condições de Funcionamento

Parâmetro	Especificação
Temperatura de Funcionamento	0C a +80C
Temperatura de Armazenamento	-20C a +60C
Gama de Humidade	20% a 80% RH (sem condensação)
Ambiente de Instalação	Apenas para utilização em interiores

Biblioteca Python

A nossa biblioteca Python fornece uma interface completa para controlar os Servomotores da Série M17. A biblioteca trata de todos os protocolos de comunicação de baixo nível e das conversões de unidades, facilitando a integração do controlo de servomotores nos seus projetos Python.

Instalação

Instale a biblioteca servomotor utilizando o pip:

```
pip3 install servomotor
```

Exemplo: Controlo Básico do Motor

O exemplo seguinte demonstra como ligar a um servomotor, ativar os MOSFETs e executar um movimento trapezoidal:

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X'                               # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Serial device path; change if needed (e.g., "COM3" on
                                          #  Windows)
DISPLACEMENT_ROTATIONS = 1.0              # 1 rotation
DURATION_SECONDS = 1.0                    # 1 second
DELAY_MARGIN = 0.10                       # +10% wait margin because the motor's clock is not
                                          #  perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
                                          #  should prompt you for the serial port or it will use
                                          #  the last used port from a file
servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Características Principais

- Suporte para todos os comandos do motor
- Conversão automática de unidades para tempo, posição, velocidade, aceleração, corrente, tensão e temperatura
- Tratamento de erros e monitorização de estado
- Suporte para vários motores no mesmo barramento
- Comece a trabalhar com apenas algumas linhas de código

Documentação

Para a documentação completa da API e mais exemplos, visite:
https://github.com/tomrodinger/servomotor/tree/main/python_programs

Biblioteca Arduino

A biblioteca Arduino para os Servomotores da Série M17 fornece uma interface fácil de utilizar para controlar motores a partir de placas Arduino. A biblioteca suporta comunicação série por hardware e inclui todas as funções essenciais de controlo do motor.

Instalação

Instale a biblioteca através do Gestor de Bibliotecas do Arduino:

- 1. Abra o Arduino IDE
- 2. Vá a Sketch → Incluir Biblioteca → Gerir Bibliotecas
- 3. Pesquise por "Servomotor"
- 4. Encontre a biblioteca chamada "Servomotor by Gearotons" e clique em Instalar



Instalação Manual

Em alternativa, se for um utilizador avançado, pode obter o código da biblioteca Arduino aqui: https://github.com/tomrodinger/Servomotor_Arduino_Library

Exemplo: Controlo Básico do Motor

Este exemplo mostra como inicializar um motor, ativar os MOSFETs e executar um movimento trapezoidal:

```
// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
//  system reset -> enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds)
//  -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.
// - Every command is checked with getError(). No command method returns a success
//   flag, and a motor that has latched a fatal error silently ignores everything
//   afterwards, so an unchecked sketch just stops moving with no symptom at all.
//   See "Checking for errors" in the API documentation.

#include <Servomotor.h>

#define ALIAS 'X'                   // Device alias
#define BAUD 230400                 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f       // 1 second
#define TOLERANCE_PERCENT 10        // +10% wait margin because the motor's clock is not
                                    //  perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))
#define POST_RESET_WAIT_MS 1500     // Keep the bus SILENT this long after a reset: the motor
                                    //  boots through a bootloader window, and any packet that
                                    //  arrives during it pins the motor in the bootloader.

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4             // TX pin to RS485 transceiver
#define RS485_RXD 5             // RX pin from RS485 transceiver
#endif

// Returns true (and explains itself) if the previous command did not succeed.
// getError() is 0 on success, positive for a fatal error reported by the motor
// (look the number up in the Error Codes section of the documentation), and
// negative for a communication failure such as -1 = no reply within 1 second.
bool failed(Servomotor &motor, const char *what) {
  int e = motor.getError();
  if (e == 0) return false;
  Serial.print("[FAIL] ");
  Serial.print(what);
  Serial.print(" -> getError() = ");
  Serial.print(e);
  Serial.println(e > 0 ? "  (motor fatal error - see the Error Codes section)"
                       : "  (communication failure - check wiring, alias and power)");
  return true;
}

void setup() {
  Serial.begin(115200); // Console serial for debugging
                        // On ESP32-S3, set Tools > USB CDC On Boot > Enabled or this
                        // output never reaches the USB serial monitor.

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time.
  // These are host-side only - they send nothing to the motor.
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  // Start from a known-clean state: this also clears any fatal error left over
  // from a previous run, which is the usual reason a motor "stops working".
  motor.systemReset();
  if (failed(motor, "systemReset")) return;
  delay(POST_RESET_WAIT_MS);

  motor.enableMosfets();
```

```
  if (failed(motor, "enableMosfets")) return;

  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  if (failed(motor, "trapezoidMove")) return;

  delay(WAIT_MS);

  // A successful move command only means the move was accepted and queued. Faults
  // such as a stall or a position-deviation trip happen later, while it executes,
  // so check the motor's own status once the motion should be finished.
  getStatusResponse status = motor.getStatus();
  if (failed(motor, "getStatus")) return;
  if (status.fatalErrorCode != 0) {
    Serial.print("[FAIL] motor faulted during the move, error code ");
    Serial.println(status.fatalErrorCode);
    return;
  }

  motor.disableMosfets();
  if (failed(motor, "disableMosfets")) return;

  Serial.println("Move completed successfully.");
}

void loop() {
}
```

A Nossa Missão

Na Gearotons, acreditamos que o futuro pertence a quem compreende e consegue trabalhar lado a lado com a inteligência artificial e os sistemas de automação. A nossa missão é capacitar a próxima geração de inovadores, tornando a tecnologia avançada de controlo de movimento acessível, compreensível e económica para criadores, educadores e engenheiros em todo o mundo. Estamos empenhados em derrubar as barreiras que tradicionalmente separaram os estudantes e os entusiastas das ferramentas de automação de nível profissional. Ao fornecer soluções integradas que eliminam a complexidade sem sacrificar a capacidade, possibilitamos experiências de aprendizagem práticas que preparam as mentes jovens para um mundo em que a criatividade humana e a precisão das máquinas trabalham em harmonia. A nossa convicção fundamental é que todos os estudantes devem ter a oportunidade de experimentar os blocos de construção da automação moderna - da robótica e dos sistemas CNC aos instrumentos científicos. Através das nossas parcerias educativas e da nossa abordagem de código aberto, estamos a formar uma geração que não se limita a usar a tecnologia, mas que verdadeiramente a compreende e a molda. Juntos, estamos a construir a base para os inovadores que irão definir um futuro extraordinário.

Código Aberto

Acreditamos em tornar o mundo melhor através da tecnologia. O software, o firmware e os ficheiros de desenho das PCB estão disponíveis aqui:

https://github.com/tomrodinger/servomotor

Versão da Ficha Técnica: 1.11 Data de Publicação: Oct. 29, 2026

(c) 2026 Todas as especificações estão sujeitas a alteração sem aviso prévio.